

Application Documentation Structure & Examples

A practical docs-as-code reference for product specs, architecture, flows, APIs, and AI coding agents

Recommended principle

Keep Markdown, Mermaid, and OpenAPI files in Git as the canonical project knowledge. Generate PDFs or websites from those sources when you need a shareable presentation.

Reference edition - 2026

1. Recommended repository structure

This structure separates product intent, feature behavior, architecture, runtime flows, API contracts, and agent instructions. It is intentionally small enough for a solo or two-person engineering team while remaining scalable.

```

project/
├── README.md
├── AGENTS.md
├── docs/
│   ├── index.md
│   ├── product/
│   │   ├── vision.md
│   │   ├── requirements.md
│   │   ├── roadmap.md
│   │   └── glossary.md
│   ├── specs/
│   │   ├── authentication.md
│   │   ├── user-profile.md
│   │   └── notifications.md
│   ├── architecture/
│   │   ├── overview.md
│   │   ├── system-context.md
│   │   ├── data-flow.md
│   │   └── decisions/
│   │       ├── 001-state-management.md
│   │       └── 002-database.md
│   ├── flows/
│   │   ├── authentication.md
│   │   ├── onboarding.md
│   │   └── payment.md
│   ├── api/
│   │   └── openapi.yaml
│   └── ai/
│       ├── context.md
│       ├── architecture-rules.md
│       └── coding-rules.md
├── src/
└── tests/
  
```

Keep it lean

Do not create empty files for every possible category on day one. Start with `index.md`, `vision.md`, `requirements.md`, `roadmap.md`, `architecture/overview.md`, one feature spec, and `AGENTS.md`. Add the rest when they have real content.

2. What belongs in each folder

Path	Role	Typical content
docs/index.md	Navigation	Entry point to all project documentation. Link to product, architecture, specs, flows, and API documents.
docs/product/	Why and what	Vision, goals, requirements, roadmap, terminology, scope, and product constraints.
docs/specs/	Feature behavior	One specification per meaningful feature: requirements, rules, states, errors, API dependencies, and acceptance criteria.
docs/architecture/	How the system is built	System boundaries, components, data movement, dependencies, deployment assumptions, and architecture diagrams.
architecture/decisions/	Why a technical choice exists	Architecture Decision Records (ADRs). Keep the decision, context, alternatives, trade-offs,

Path	Role	Typical content
		and consequences.
docs/flows/	Runtime/user journeys	Mermaid flowcharts and sequence diagrams for user journeys and multi-system interactions.
docs/api/	Machine-readable contracts	OpenAPI specification and API-specific conventions. Prefer the contract itself over duplicated endpoint prose.
docs/ai/	Stable agent context	Project context and rules that AI coding tools should read before making implementation decisions.
AGENTS.md	Agent operating instructions	Repository-level instructions: what to read, implementation constraints, testing expectations, and definition of done.

3. Documentation ownership model

Information	Canonical source	Do not duplicate into
Product direction	docs/product/roadmap.md	A second detailed task backlog
Current work status	GitHub Issues / Projects	roadmap.md line-by-line status
Feature behavior	docs/specs/<feature>.md	Scattered implementation comments
API schema	docs/api/openapi.yaml	Manually maintained endpoint tables
Technical decisions	architecture/decisions/*.md	Chat history or undocumented PR comments
Implementation rules	AGENTS.md + docs/ai/	Repeated prompts pasted into every AI session

Source-of-truth rule

A fact should have one authoritative home. Other documents should link to it instead of copying it. This reduces stale documentation and conflicting instructions.

4. Sample: docs/index.md

Use this file as the documentation home page. It should be short and navigational rather than a second README.

```

MARKDOWN
# Project Documentation

## Product

- [Vision](product/vision.md)
- [Requirements](product/requirements.md)
- [Roadmap](product/roadmap.md)
- [Glossary](product/glossary.md)

## Engineering

- [Architecture Overview](architecture/overview.md)
- [System Context](architecture/system-context.md)
- [Architecture Decisions](architecture/decisions/)

## Feature Specifications

- [Authentication](specs/authentication.md)
- [User Profile](specs/user-profile.md)

## Flows

- [Authentication Flow](flows/authentication.md)
- [Onboarding Flow](flows/onboarding.md)

## API

- [OpenAPI Contract](api/openapi.yaml)

```

```
## AI / Agent Context

- [Project Context](ai/context.md)
- [Architecture Rules](ai/architecture-rules.md)
- [Coding Rules](ai/coding-rules.md)
```

5. Sample: docs/product/vision.md

```
MARKDOWN
# Product Vision

## Problem

Users need a simple way to <describe the problem>.

## Target Users

- Primary: <user type>
- Secondary: <user type>

## Product Goal

Build a mobile application that <one-sentence outcome>.

## Success Criteria

- Users can complete the primary workflow in under 2 minutes.
- Critical flows work reliably on supported Android and iOS versions.
- The MVP validates <business assumption>.

## Non-Goals

The MVP will not include:

- Advanced analytics
- Admin automation
- Recommendation engine
```

6. Sample: docs/product/requirements.md

```
MARKDOWN
# Product Requirements

## Functional Requirements

### FR-001 Authentication
The user can sign in using email and password.

### FR-002 Persistent Session
The authenticated session persists after application restart until it expires or the user logs out.

### FR-003 Profile
The user can view and edit allowed profile fields.

## Non-Functional Requirements

### NFR-001 Performance
Primary screens should become interactive within the agreed performance budget.

### NFR-002 Security
Authentication tokens must not be written to logs.

### NFR-003 Reliability
Network failures must produce a recoverable UI state.

## Constraints

- Mobile platforms: Android and iOS
- Backend contract: REST/JSON
- API changes must update docs/api/openapi.yaml
```

7. Sample: docs/product/roadmap.md

Keep the roadmap high level. GitHub Issues and Projects should track executable work.

```
MARKDOWN
# Roadmap

## MVP

- [ ] Authentication
- [ ] User profile
- [ ] Home experience
- [ ] Core notifications

## V1

- [ ] Social login
- [ ] Search
- [ ] Push notification preferences

## Later

- [ ] Recommendations
- [ ] Analytics
- [ ] Advanced admin tooling

## Explicitly Out of Scope for MVP

- Multi-tenant administration
- Offline-first synchronization
```

8. Sample: docs/specs/authentication.md

A feature spec should define externally visible behavior and acceptance criteria without becoming an implementation dump.

```
MARKDOWN
# Authentication

## Goal

Allow a registered user to authenticate securely and enter the application.

## Requirements

- User can enter email and password.
- User can submit credentials.
- A valid session navigates to Home.
- Invalid credentials display a useful error.
- Session persists according to the authentication policy.

## Business Rules

- Email is normalized before submission.
- Submit is disabled while a login request is in progress.
- Authentication errors do not expose backend internals.

## UI / State Model

- idle
- submitting
- authenticated
- invalidCredentials
- networkFailure

## API Dependency

`POST /auth/login`

See: `../api/openapi.yaml`

## Error Cases

| Case | Expected behavior |
| --- | --- |
```

```
| Invalid credentials | Show login error and keep the form editable |
| No network | Show retryable network state |
| Server error | Show generic failure state |
```

Acceptance Criteria

- [] Valid credentials open Home.
- [] Invalid credentials never open Home.
- [] A double tap cannot submit duplicate requests.
- [] Network failure can be retried.
- [] No authentication token appears in logs.

9. Sample: docs/flows/authentication.md

Use Mermaid for diagrams so the flow remains diffable, reviewable, and editable by AI agents.

```
MARKDOWN
# Authentication Flow

```mermaid
flowchart TD
 A([Launch App]) --> B{Valid session?}

 B -->|Yes| C[Home]
 B -->|No| D[Login]

 D --> E[Enter email and password]
 E --> F[Submit]
 F --> G{Credentials valid?}

 G -->|Yes| H[Store session]
 H --> C
 G -->|No| I[Show error]
 I --> D
```

## Notes

- Session validation happens before routing to Home.
- The API contract is defined in `../api/openapi.yaml`.
- Behavioral requirements are defined in `../specs/authentication.md`.
```

Sequence diagram variant

```
MARKDOWN
```mermaid
sequenceDiagram
 actor User
 participant App
 participant API
 participant DB

 User->>App: Submit credentials
 App->>API: POST /auth/login
 API->>DB: Find user
 DB-->>API: User record

 alt Valid credentials
 API-->>App: 200 + session
 App-->>User: Open Home
 else Invalid credentials
 API-->>App: 401
 App-->>User: Show error
 end
```
```

10. Sample: docs/architecture/overview.md

```
MARKDOWN
```

```
# Architecture Overview

## Goals

- Keep feature boundaries clear.
- Keep business behavior testable.
- Keep backend contracts explicit.
- Avoid framework-specific abstractions unless they solve a current need.

## System Diagram

```mermaid
graph LR
 User[User] --> App[Mobile App]
 App -->|HTTPS / JSON| API[Backend API]
 API --> DB[(PostgreSQL)]
 API --> Push[Push Provider]
```

```
...

Application Layers

1. Presentation - screens, widgets, navigation, state orchestration.
2. Domain/Application - use cases and business rules when needed.
3. Data - repositories, remote/local data sources, DTO mapping.

Dependency Rule

Higher-level business behavior should not depend directly on UI framework details.

Related Decisions

- [ADR-001 State Management](decisions/001-state-management.md)
- [ADR-002 Database](decisions/002-database.md)
```

## 11. Sample: architecture decision record

Create an ADR when a decision is important enough that a future developer may ask, "Why did we choose this?"

```
MARKDOWN
ADR-001: State Management

- Status: Accepted
- Date: 2026-08-27

Context

The application needs predictable event-driven state transitions and testable asynchronous workflows.

Decision

Use BLoC for feature state management. Feature events and states are modeled explicitly.

Alternatives Considered

Cubit
Simpler for small state changes, but it does not match the project's event-driven convention.

Global mutable service state
Rejected because state transitions become implicit and harder to test.

Consequences

Positive
- Explicit event history
- Predictable testing model
- Consistent feature structure

Negative
- More boilerplate than simpler state containers
- Requires disciplined event/state naming
```

## 12. Sample: docs/api/openapi.yaml

Use OpenAPI as the contract. Feature specs should link to operations instead of rewriting request and response schemas.

```
YAML
openapi: 3.1.0
info:
 title: Application API
 version: 1.0.0

paths:
 /auth/login:
 post:
 operationId: login
 summary: Authenticate a user
 requestBody:
 required: true
 content:
 application/json:
 schema:
 type: object
 required: [email, password]
 properties:
 email:
 type: string
 format: email
 password:
 type: string
 format: password
 responses:
 '200':
 description: Authenticated
 '401':
 description: Invalid credentials
```

## 13. Sample: docs/ai/context.md

```
MARKDOWN
AI Project Context

Product

This repository contains the mobile client for <application>.

Priorities

1. Correctness
2. Maintainability
3. Clear user-facing behavior
4. Performance where measured or required

Documentation Sources

Before changing a feature, read:

1. `docs/specs/<feature>.md`
2. Relevant files under `docs/flows/`
3. `docs/architecture/overview.md`
4. Relevant ADRs
5. `docs/api/openapi.yaml` for backend contracts

Rules

- Do not invent an API contract.
- Do not add a dependency without a concrete reason.
- Prefer existing project patterns over new abstractions.
- If implementation behavior changes, update the corresponding spec.
- If a significant architecture decision changes, add or supersede an ADR.
```

## 14. Sample: AGENTS.md

```

MARKDOWN
Agent Instructions

Before Editing Code

Read the relevant documentation first:

1. `docs/architecture/overview.md`
2. The feature specification under `docs/specs/`
3. Relevant flow diagrams under `docs/flows/`
4. Relevant ADRs under `docs/architecture/decisions/`
5. `docs/api/openapi.yaml` when API behavior is involved

Implementation Rules

- Follow established architecture and naming conventions.
- Do not introduce new dependencies unless required.
- Keep changes scoped to the requested behavior.
- Add or update tests for changed behavior.
- Never silently change API assumptions.

Documentation Rules

Update documentation when:

- externally visible feature behavior changes
- a flow changes
- an API contract changes
- an architecture decision changes

Definition of Done

A change is complete when:

- implementation is complete
- relevant tests pass
- acceptance criteria are satisfied
- documentation matches the implementation

```

## 15. Recommended workflow

```

graph TD
 Idea --> PVR[Product vision / requirement]
 PVR --> FS[Feature specification]
 FS --> USF[User/system flow (Mermaid)]
 USF --> AD[Architecture decision (only if needed)]
 AD --> API[API contract (OpenAPI)]
 API --> GI[GitHub Issue / implementation plan]
 GI --> CT[Code + tests]
 CT --> RAC[Review acceptance criteria]
 RAC --> UAD[Update affected documentation]

```

### Avoid documentation ceremony

Not every bug fix needs a new spec or ADR. Update documentation only when the documented contract, behavior, flow, or architectural reasoning actually changes.

## 16. Minimum viable documentation set

For a new app, you can start with only these files and expand when necessary:

```

README.md
AGENTS.md
docs/
├── index.md
├── product/
│ ├── vision.md
│ ├── requirements.md
│ └── roadmap.md
├── specs/
│ └── <first-feature>.md
├── architecture/
│ └── overview.md
├── flows/
│ └── <first-feature>.md
├── api/
│ └── openapi.yaml

```

- Markdown files are the canonical human-readable source.
- Mermaid inside Markdown is the canonical source for most engineering diagrams.
- OpenAPI is the canonical API contract.
- GitHub Issues/Projects track executable work and current status.
- PDF is a generated sharing format, not the primary engineering source.

## 17. Documentation quality checklist

Check	Question
Single source of truth	Is this fact defined in one authoritative place?
Discoverability	Can a new engineer find the relevant document from docs/index.md?
Behavior vs implementation	Does the feature spec describe observable behavior rather than low-level code?
Diagram maintainability	Is the diagram represented as Mermaid text when practical?
Agent usability	Can an AI agent identify the required spec, flow, API contract, and architecture rules?
Change discipline	Will a code behavior change trigger an update to the affected documentation?
No duplicate backlog	Is current task status kept in GitHub Issues/Projects instead of duplicated in docs?